

# **Tema 1**

## **Introducción a la programación en red**

Humberto Martínez Barberá  
Daniel Sánchez Martínez

# Contenidos

- Comunicaciones y sus retos en la actualidad
- Arquitecturas de aplicaciones en red
  - cliente/servidor
  - *peer-to-peer*
- Modelos de programación
  - comunicación no persistente vs persistente
  - procedimientos y objetos distribuidos
  - servicios web
- Seguridad

# Introducción (1)

- En la actualidad las comunicaciones forman parte de nuestra vida diaria
  - trabajo, ocio, información, etc
  - uso prominente de Internet
- Convergencia de servicios en Internet
  - datos, voz y vídeo sobre redes TCP/IP
  - nuevos servicios y tecnologías aparecen continuamente
    - *cloud computing*
    - *green communications*
    - *Internet of Things (IoT)*

# Introducción (2)

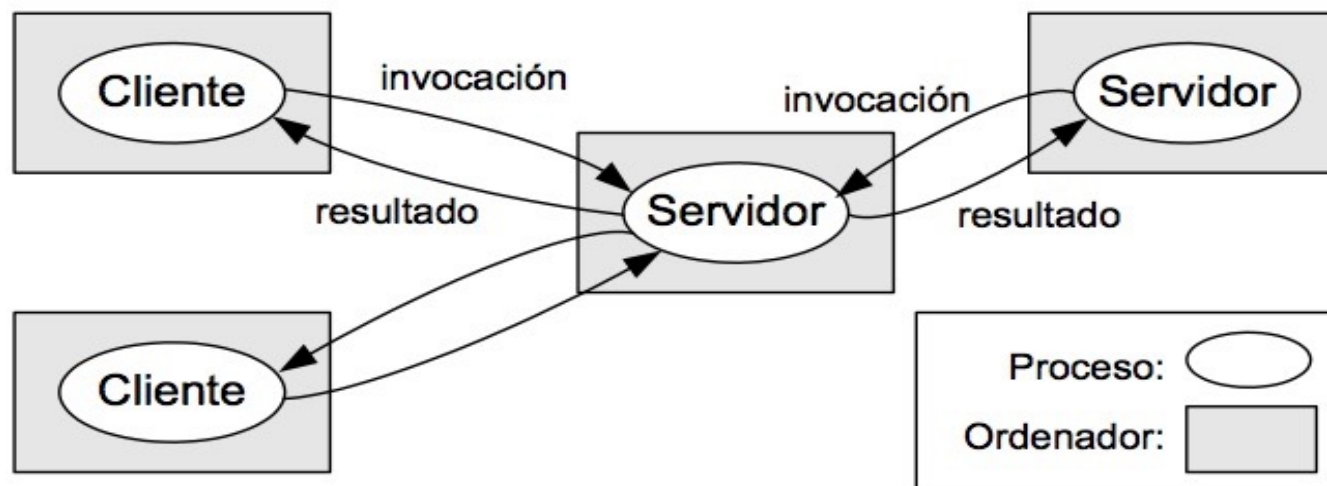
- Diferentes redes de acceso con las que consumimos los mismos servicios
  - cableadas: Ethernet, ADSL y fibra principalmente
  - inalámbricas: WiFi y UMTS principalmente
  - muy diferentes en cuanto a prestaciones, aunque los usuarios demandan rendimientos similares
  - diversos dispositivos con distintas características
- Transacciones requieren un elevado nivel de seguridad
  - comercio electrónico
  - administración electrónica
  - banca electrónica

# Retos

- Proporcionar tecnologías de comunicación...
  - que den soporte a la creciente gama de servicios que se despliegan en Internet
  - que se adecuen a las prestaciones que ofrecen las redes de acceso comunes
  - que se adapten a los distintos dispositivos de acceso (*PCs, laptops, tablets, PDAs, smartphones, etc*)
  - que sean eficientes en términos de consumo energético y de ancho de banda
  - que proporcionen un adecuado nivel de seguridad
- Necesario conocer en profundidad la pila TCP/IP y cómo programar en ella

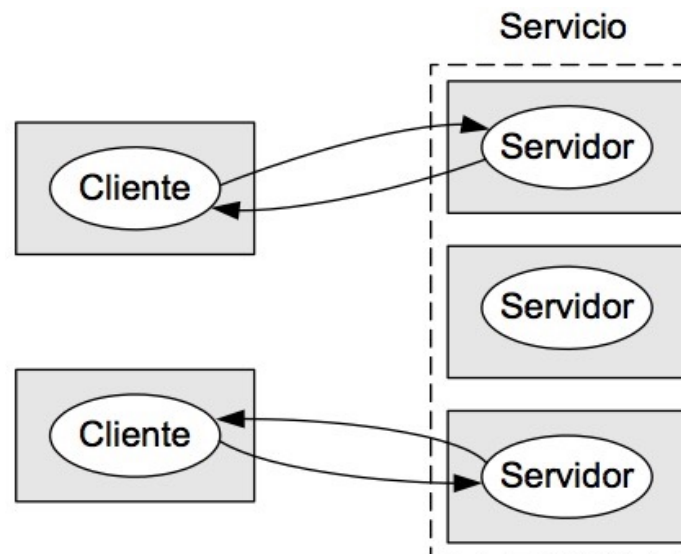
# Modelo cliente/servidor (1)

- Los procesos cliente interactúan con un proceso servidor
- modelo más habitual, la mayoría de servicios en Internet lo implementa (HTTP, SMTP, FTP, DNS, etc)
- cliente inicia la comunicación con el servidor
- un servidor puede actuar como cliente de otro servidor



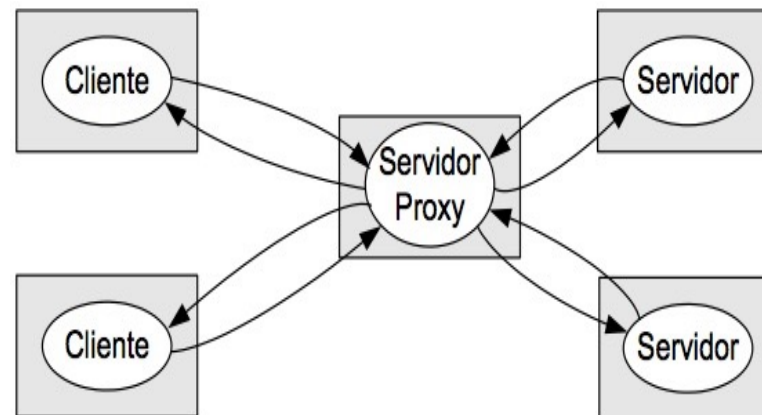
# Modelo cliente/servidor (2)

- Un mismo servicio puede estar **distribuido** en múltiples *hosts*
- *cluster* de servidores que gestionan una réplica de los mismos recursos
  - tolerancia a fallos, balanceo de carga, etc
- servidores que gestionan subconjuntos del total de los recursos
  - reducción de latencia según proximidad geográfica, etc



# Modelo cliente/servidor con *proxies*

- Un **proxy caché** es un nodo intermedio entre la comunicación cliente/servidor
- almacena **copias** de los recursos del servidor
- define un mecanismo de **actualización** de las copias
- mejora **latencia** del servicio, aunque puede devolver recursos desactualizados
- actúa simultáneamente como **servidor** (cliente ↔ proxy) y **cliente** (proxy ↔ servidor)
- Ej: proxy web (HTTP)

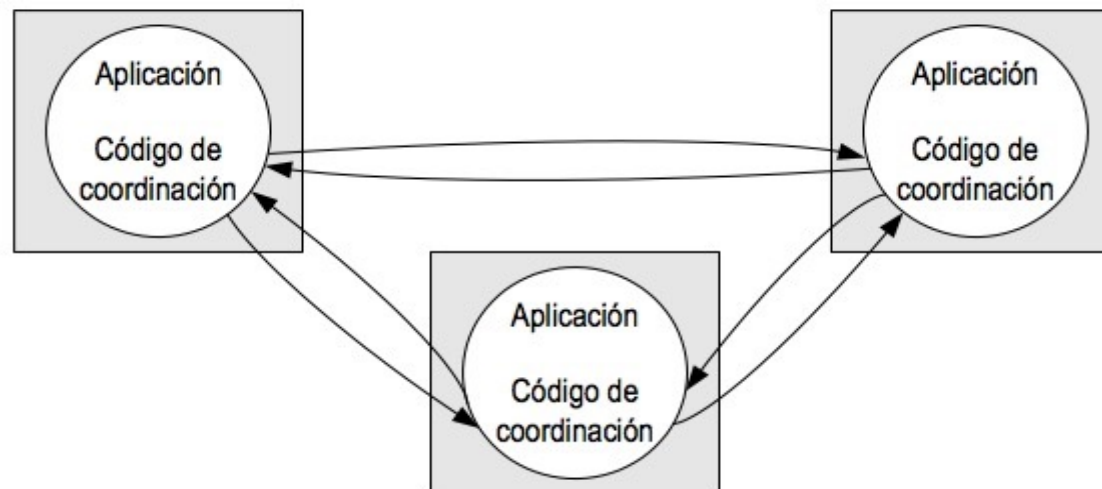


# Tipos de proxy caché

- **Transparente**
  - cliente no es consciente de la presencia del proxy
  - no requiere configuración del cliente
- **No transparente**
  - cliente es consciente de la presencia del proxy
  - requiere configuración del cliente

# Modelo *peer-to-peer*

- Todos los procesos juegan el mismo papel
- comunicación “igual a igual”
- en determinados contextos, al elemento que inicia la comunicación se le sigue llamando “cliente” y al que responde “servidor”



# Comunicación no persistente

- Las **comunicación no persistente** (*transient*) requiere que emisor y receptor estén activos al tiempo que se produce el intercambio de mensajes
- **acoplamiento temporal** entre emisor y receptor
- Este servicio lo proporciona el nivel de transporte y lo exporta a las aplicaciones por medio de **sockets**
- puntos de entrada/salida para procesos que se comunican a través de una red

# APIs de sockets (1)

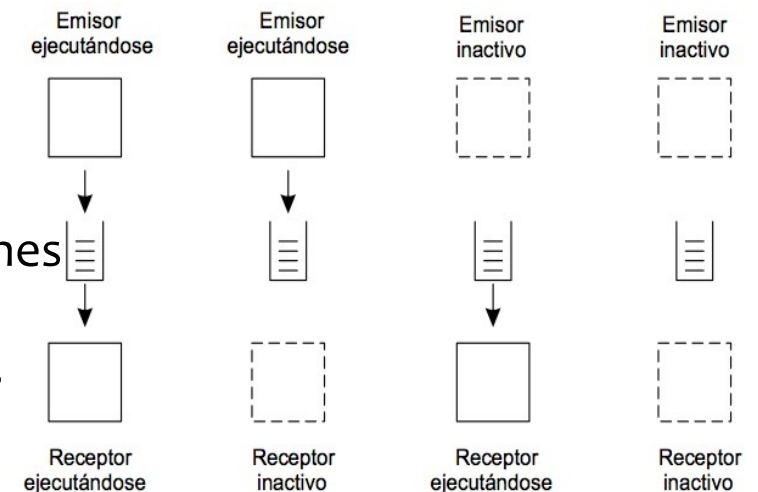
- Bajo nivel
  - nos permiten un mayor control de las comunicaciones
  - **BSD (POSIX) sockets**
    - lenguaje C
    - interfaz con protocolos de transporte, red y nivel de enlace
    - Linux, Mac Os X, iOS, Solaris, HP-UX, FreeBSD, OpenBSD, etc
  - WinSock (*Windows Sockets API, WSA*)
    - lenguaje C/C++
    - basado en sockets BSD
    - sistemas Windows

# APIs de sockets (2)

- Alto nivel
  - mayor facilidad de programación a costa de menor potencia
  - se implementan usando APIs de bajo nivel
  - Java sockets
    - lenguaje Java
    - interfaz con protocolos de transporte
    - Multiplataforma
- Interfaces con protocolos de aplicación
  - ej: CFNetwork y Foundation networking APIs
  - lenguaje Objective-C
  - plataforma iOS (iPhone, iPad)

# Comunicación persistente

- La **comunicación persistente** se basa en el uso de **colas** (buffers) de mensajes que se sitúan entre el emisor y el receptor
- **desacoplamiento temporal** entre emisor y receptor
- Servicio a **nivel de aplicación** proporcionado por un **middleware de mensajería**
- proporciona un API al desarrollador de aplicaciones para enviar/recibir mensajes que se introducen/extraen de las colas por medio de los **gestores de colas**
- se implementa por medio de sockets
- implementaciones: Java Message Service (JMS), etc

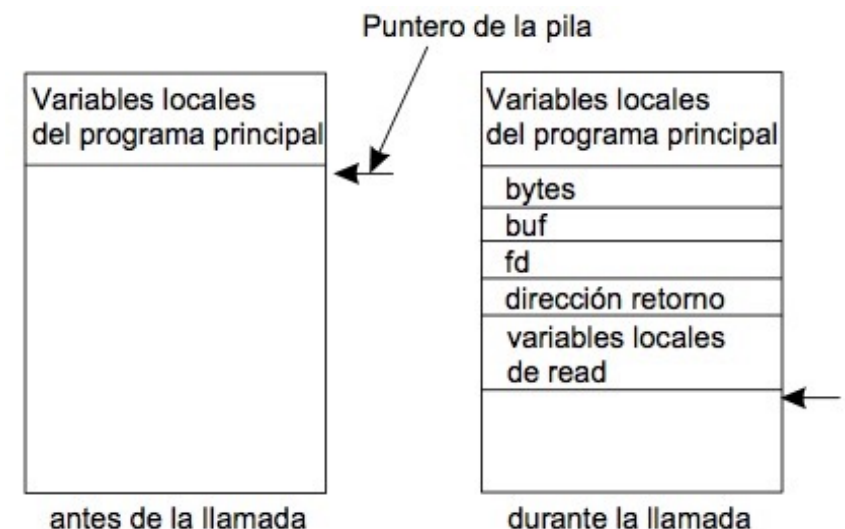


# Procedimientos y objetos remotos

- Aplicaciones distribuidas
  - procedimientos y objetos locales
    - varios programas se comunican usando APIs de red (sockets, colas de mensajes)
    - ej: cliente y servidor HTTP, dos programas en hosts distintos para implementar la aplicación web
  - procedimientos y objetos remotos
    - un programa invoca procedimientos o métodos de objetos que se alojan en un host distinto
    - ej: escalabilidad de aplicaciones intensivas en procesamiento, exportar la funcionalidad de una aplicación a otras por medio de un API remoto, etc

# Llamada a procedimiento local

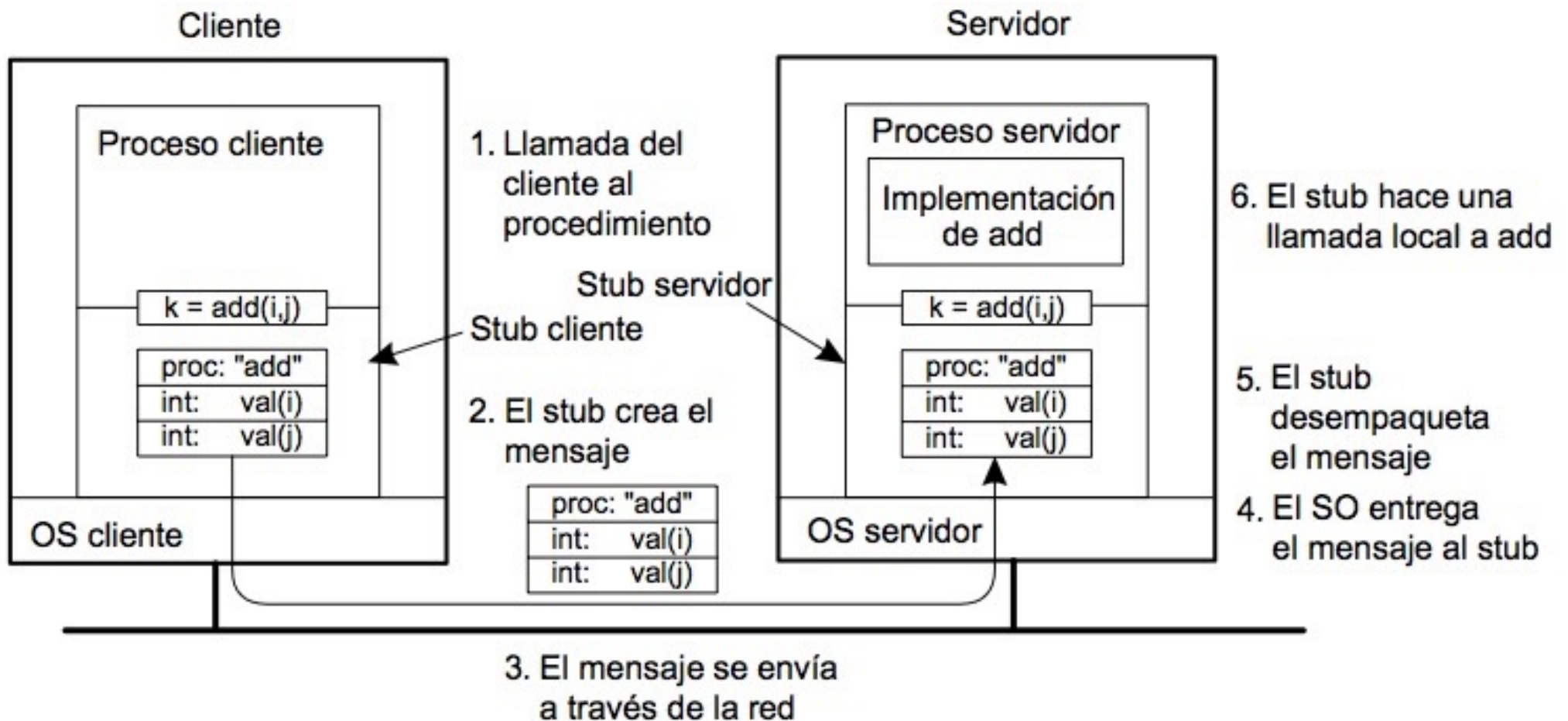
- Se actualiza la pila
- *push* (parámetros del procedimiento)
- *push* (dirección de retorno)
- El *linker* resuelve la dirección donde se halla el código del procedimiento y salta a esta dirección
- Al finalizar la ejecución del procedimiento
  - *jump (pop (dirección de retorno))*
  - *pop* (parámetros del procedimiento)



# Llamada a procedimiento remoto

- “Remote Procedure Call” (RPC)
- Cuando el procedimiento es remoto, el *linker* invoca lo que se conoce como **stub de cliente**
- El stub de cliente introduce los parámetros de la llamada en un mensaje que envía al servidor
- En el servidor, el mensaje llega al **stub de servidor**, que obtiene los parámetros e invoca a un procedimiento local
- El stub de servidor obtiene el valor de retorno, lo encapsula en un mensaje, y lo envía al cliente
- En el cliente, el mensaje llega al stub de cliente que devuelve el valor de retorno al proceso que hizo la llamada RPC
- La comunicación se implementa por medio de sockets
- RPC nos abstrae de los detalles de comunicación

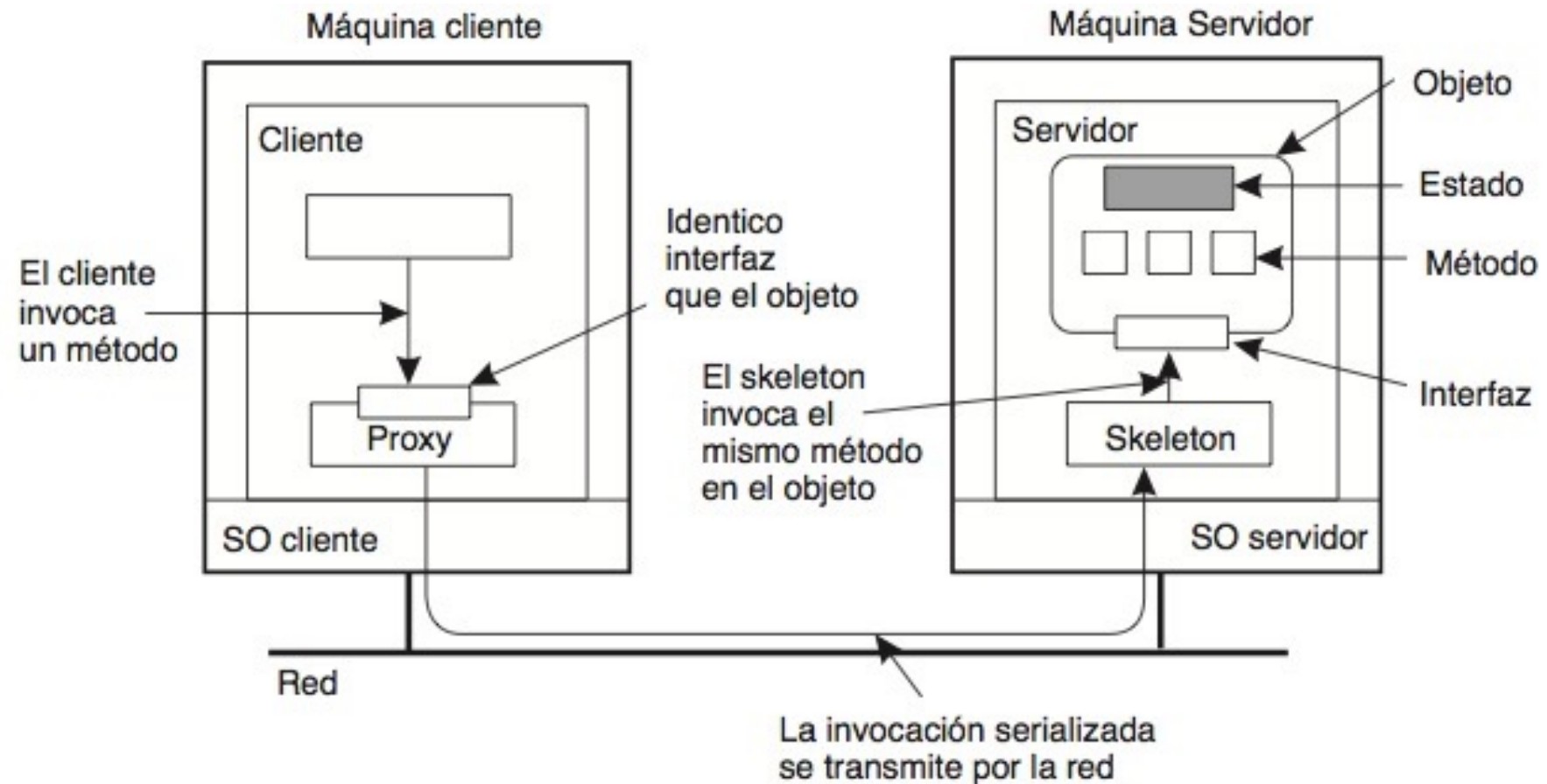
# Llamada a procedimiento remoto (cont)



# Llamada a método remoto

- “Remote Method Invocation” (RMI)
- Sistemas de objetos distribuidos en distintas máquinas
- Mismo concepto que en RPC, sólo que ahora invocamos métodos en objetos remotos
- Cada objeto remoto tiene un **proxy** local (análogo al stub de cliente)
- El proxy genera el mensaje que se envía al servidor que aloja el objeto
- El **skeleton** (análogo al stub de servidor) recibe el mensaje e invoca el método correspondiente en el objeto local
- El resultado se devuelve al proxy para que lo entregue al llamador
- La comunicación se implementa por medio de sockets
- RMI nos abstrae de los detalles de comunicación

# Llamada a método remoto (cont)



# Paso de parámetros (1)

- Las implementaciones RPC y RMI deben resolver el problema del **paso de parámetros**
- los hosts tienen distintos formatos de representación interna (*little endian* vs *big endian*)
- los hosts usan distintos tamaños de tipos básicos (32 bits vs 64 bits)
- los hosts usan distintos formatos de tipos básicos (cadenas, números en punto flotante)
- Problema: un parámetro codificado en un host puede no entenderse según la codificación de otro host

## Paso de parámetros (2)

- Solución: convertir de **representación interna** a una **representación externa** común
- ej: *eXternal Data Representation* (XDR) de Sun, usado con ONC RPC
- *marshalling* (empaquetado o serialización):  
interna → externa
- *unmarshalling* (desempaquetado o serialización):  
externa → interna

# Paso de parámetros por referencia

- Problema: cuando un parámetro se pasa por **referencia** se hace alusión a un área de memoria local
- Solución: usar paso por **copia/devolución** de la memoria referenciada
- se copia el valor de la memoria en la llamada al stub de cliente
- en el stub de servidor se hace la llamada al procedimiento o método local (usando paso por referencia)
- el valor de la memoria tras la llamada se comunica al stub de cliente
- El proceso anterior se puede optimizar sabiendo si el parámetro es sólo de entrada o de salida, ¿cómo?

# Stubs de cliente y servidor

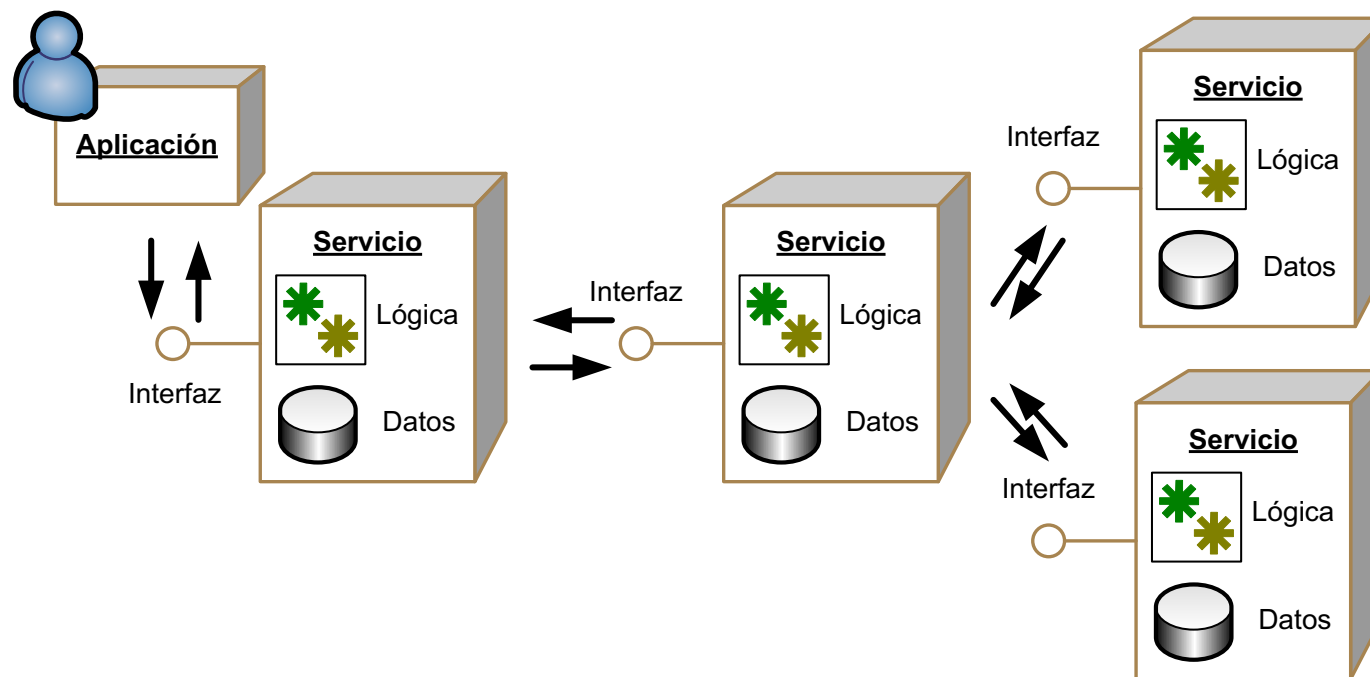
- Los stubs de cliente y servidor se encargan de
- marshalling/unmarshalling de parámetros (ej: XDR)
- localizar el procedimiento u objeto remoto por medio de un servicio centralizado (ej: portmapper)
- detalles de comunicación TCP/UDP
- Los stubs son generados automáticamente por medio de un compilador (ej: rpcgen) a partir de una descripción que proporciona el programador

# APIs de procedimientos y objetos remotos

- Procedimientos remotos (programación imperativa)
  - *Sun Open Network Computing (ONC) RPC, lenguaje C*
  - *Open Software Foundation (OSF) Distributed Computing Environment (DCE) RPC, lenguaje C*
- Objetos remotos (programación orientada a objetos)
  - *Java Remote Method Invocation (RMI), lenguaje Java*
  - *ZeroC Internet Communication Engine (ICE), multi-lenguaje (C++, Java, Python, Ruby, etc)*
  - *Common Object Request Broker Architecture (CORBA), múltiples implementaciones, multi-lenguaje*

# Servicios Web

- SOA: Services Oriented Architecture.
  - Servicios interoperables.
  - Extensibilidad e integración con aplicaciones distribuidas.



# Servicios Web

## ■ Principios

- Bajo acoplamiento
- Abstracción
- Reusabilidad

## ■ Roles

- Proveedor del servicio
- Consumidor del servicio

## ■ Tecnologías que implementan SOA

- SOAP, RPC
- REST
- DCOM
- CORBA

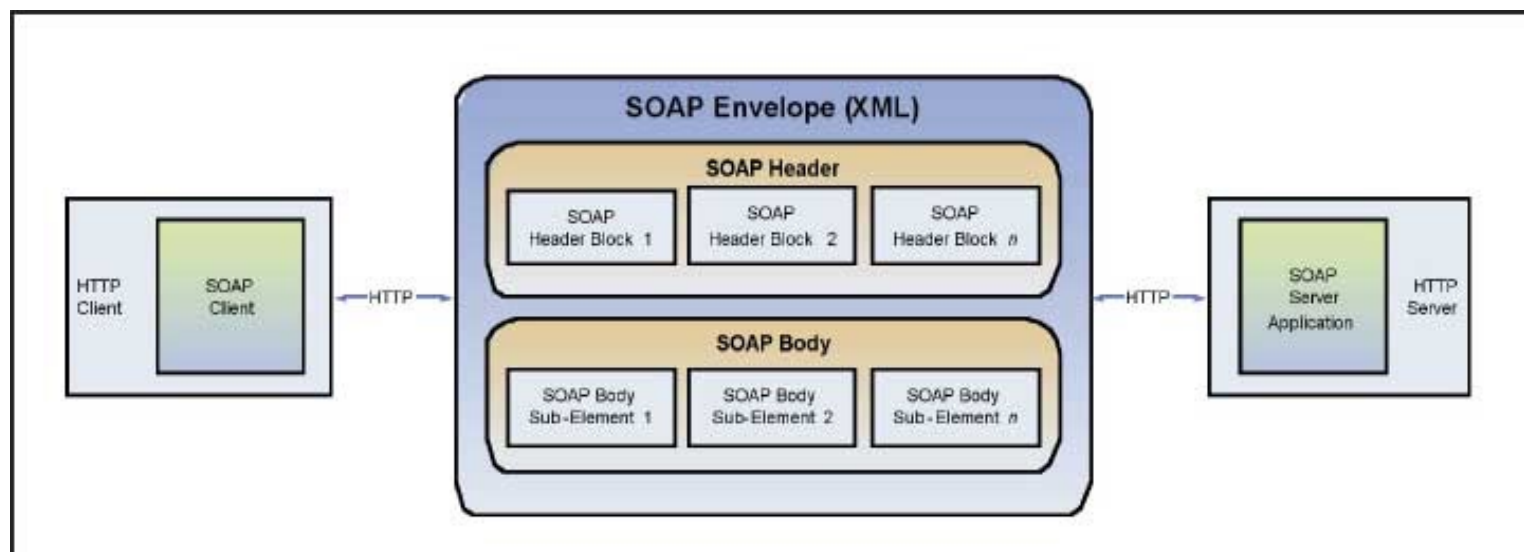
# SOAP

## ■ Principios

- Contrato de servicio
  - Fuertemente tipado
- Protocolo que define operaciones y mensajes
- Infraestructura basada en el uso de XML
- Fácil de consumir
- Existen multitud de herramientas de desarrollo

## ■ Estándares

- WSDL: Web Service Description Language
- UDDI: Universal Description, Discovery and Integration
- Puede construirse sobre HTTP o SMTP
- WS-\*



# REST



## Principios



Se construye sobre el protocolo HTTP



Operaciones POST, GET, DELETE y PUT



No agrega ninguna capa adicional



Cada URL representa un recurso



Utilización de tipos MIME



Ligero y con resultados legibles



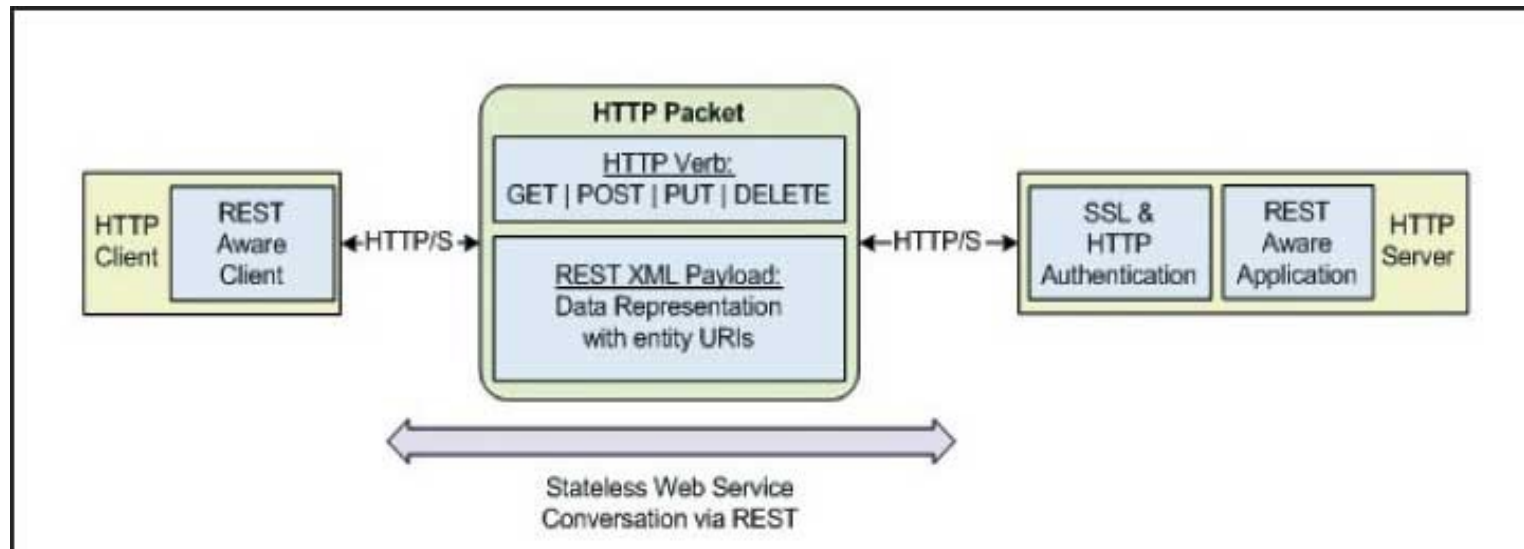
Muy escalable



Compatibilidad con proxies y componentes intermedios



No requiere de herramientas específicas



# SOAP vs REST

## ■ SOAP más adecuado cuando...

- Es necesaria una descripción muy detallada de la interfaz
- Se deben abordar requerimientos complejos sobre transacciones, seguridad y direccionamiento
- Se requiere soportar procesamiento asíncrono

## ■ REST más adecuado cuando

- No se requiere mantenimiento de estado
- La infraestructura de caching puede mejorar el rendimiento
- El productor y el consumidor conocen los detalles de la comunicación
- El ancho de banda es limitado y/o los dispositivos tienen recursos escasos

# APIs de Servicios Web

- Apache CXF
- Apache Axis2
- Sun JAX-WS

# Seguridad

 Servicios electrónicos

- Ventajas para los usuarios y organizaciones
  - Rapidez, comodidad y eficiencia
- Riesgos → medidas de seguridad
  - Frenar ataques a las redes y a la información confidencial: intrusión, suplantación de identidad, acceso a datos confidenciales, etc.
  - Infraestructura para la confianza

## ■ Objetivos de la Seguridad de la Información

- Confidencialidad
- Integridad
- Disponibilidad
- Uso autorizado
- La seguridad total NO existe
- Tiempo y recursos



# Seguridad

- Necesitamos proteger nuestros servicios de ataques y usos malintencionados.
  - Aplicaciones internas y externas deben poder invocarlos de una forma sencilla.
  - La interoperabilidad no puede sufrir las consecuencias.
- Debemos apartarnos del modelo tradicional
  - Cada aplicación/servicio no debe gestionar la seguridad “a su manera” → uso de estándares.
  - No podemos limitarnos a interacciones directas entre un cliente y un servidor (composición de servicios, bus de integración...)
- Puede ser implementada en diferentes capas
  - IP, transporte, aplicación...
  - Pueden ser aplicadas de forma complementaria

# Seguridad

